36. REST API Design, Middleware, Auth Basics

36. REST API Design, Middleware, Auth Basics

Phase: Phase 6 — Systems and infrastructure
Estimated time: 2 days
Slug: `36-rest-middleware-auth`

Learning objectives

- Design resource-oriented REST endpoints
- Implement middleware for logging/auth
- Use bearer tokens / basic auth patterns
- Return consistent error JSON

Prerequisites

Complete prior numbered lessons. Type every example yourself.

REST sketch

- `GET /items` list
- `GET /items/{id}` get
- `POST /items` create
- `DELETE /items/{id}` delete

Use proper status codes: 201 create, 404 missing, 400 validation, 401/403 auth.

Middleware

```
func WithRequestID(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        id := r.Header.Get("X-Request-ID")
        if id == "" { id = uuidNew() }
        ctx := context.WithValue(r.Context(), ctxKeyID, id)
        w.Header().Set("X-Request-ID", id)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}
```

Auth basics

Validate `Authorization` header; never roll crypto casually — use proven libraries. Separate authentication (who) from authorization (what allowed).

Common mistakes (especially from Python instincts)

- Leaking stack traces to clients
- Auth only on some routes by accident
- Using GET for state changes

Exercises

Work in `exercises/36-rest-middleware-auth/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/36-rest-middleware-auth/` until you have an honest failing attempt.

Self-check

1. 401 vs 403?
2. What does middleware do?
3. Why consistent error shape?

Further reading

- https://go.dev/doc/tutorial/web-service-gin